

CHALLENGE #2: No FA Write-up.

No FA

Medium

Web Exploitation

picoCTF 2026

AUTHOR: DARKRAICG492

Description

Seems like some data has been leaked! Can you get the flag?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints

123

1,710 users solved

96% Liked

picoCTF{FLAG}

Submit Flag

Solving Process:

The challenge began by opening the provided website link and reviewing the materials included with the challenge. Two important files were given: `app.py`, which contained the application’s server-side code, and `users.db`, which was the database storing user credentials. Additionally, hints were provided mentioning **salt**, **rockyou**, and **two-factor authentication (2FA)**, indicating that password hashing and authentication mechanisms would play a major role in solving the challenge.

```
(jure@kali)~$ cat app.py
from flask import Flask, render_template, request, flash, redirect, url_for, session, send_file
from dotenv import load_dotenv

import db
import os
import hashlib
import random
import time

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY')

@app.before_request
def initialize():
    app.before_request_funcs[None].remove(initialize)
    db.init_db()

@app.route("/")
def home():
    if 'username' not in session or session['logged'] == 'f':
        flash('Please login to access this page', 'red')
        return redirect(url_for('login'))

    flag = "No flag for you!!"
    if session.get('username') == 'admin':
        flag = os.getenv('FLAG')

    return render_template("index.html", flag=flag)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = db.get_user_by_username(username)
        if user and hashlib.sha256(password.encode()).hexdigest() == user['password']:
            if user['two_fa']:
                # Generate OTP
                otp = str(random.randint(1000, 9999))
                session['otp_secret'] = otp
                session['otp_timestamp'] = time.time()
                session['username'] = username
                session['logged'] = 'true'
                # send OTP to mail —
                return redirect(url_for('two_fa'))
            else:
                session['logged'] = 'true'
                return redirect(url_for('home'))
```

Unable to connect

cat users.db

sqlite_sequenceCREATE TABLE sqlite_sequence(name,seq)tableusersusersCREATE TABLE users (
id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT UNIQUE NOT NULL,
email TEXT NOT NULL,
password TEXT NOT NULL,
two_fa BOOLEAN NOT NULL DEFAULT 0
)
indexsqlite_autoindex_users_1users
melissa.thomasmelissa.thomas@nfa.com564c89c28d93e8485b76a41deca21ab28e60a32c506e479b925f4643722e9f83kbi'7*;*
kevin.andersonkevin.anderson@nfa.com8bac48021ebd453dbd876d43fa28c8e383fc16176fc8b12fa474b01eb9fa4df5m)9*
angela.martinezangela.martinez@nfa.come64b5893827166e4568af8ece105d8c0839772ae10fba3c11e77b5fb3c0ef0c6w*
john.doejohn.doe@nfa.com599a4410e2af69d1585f16d82d4b5f0abf3ad09fa42b9d55d7b7a50671ccf8c1ffff1_-*bc!1*4c880d0%5*7s
)kevin.anderson*#joseph.cole)nicole.garrett-patrick.thompson+michelle.martin#eric.harris+stephanie.white'brian.jacks
+angela.martinez
Schristopher.williams
1jennifer.rodriguez %david.garci%linda.wilson'michael.davis admin#emily.brown%robert.jones!jane.smith
users
john.

Hints

123

What makes 2FA safe?

Hints

123

rockyou rockyou rockyou

Hints

123

What makes 2FA safe?

The first step was to analyze the provided files. Using the terminal, the contents of the files were inspected. The `app.py` file contained the logic for the web application, including how the login and authentication system worked. The `users.db` file was identified as an SQLite database, which meant it could store structured information such as usernames and hashed passwords. However, simply viewing the file directly did not clearly show the stored credentials.

To properly extract the credentials, an SQLite query was executed to retrieve the usernames and password hashes from the `users` table. The query selected the `username` and `password` fields from the database. After running the query, several usernames and hashed passwords were displayed. Among them, the admin account was identified as the most important target since logging in as the administrator would likely grant access to restricted parts of the system.

```

(jure@kali) ~$ python3 -c 'import sqlite3; conn = sqlite3.connect("users.db"); [print(f'[{r[0]}: {r[1]}']) for r in conn.execute("SELECT username, password FROM users")]; conn.close()'
john.doe: 599a4410e2af6d9f585f16d82da4b5f8a9f3a09f42b9955d7b7a50671ccf8c1
john.smith: 81c68634db211e0d5632839f7efc8601743f1ef0c94da8220e26ab221e0ff1
robert.jones: aaf120fcb16e20e2d18e63e668e060b5e4a52c5e0b3f03877365fe87ca2ccdb

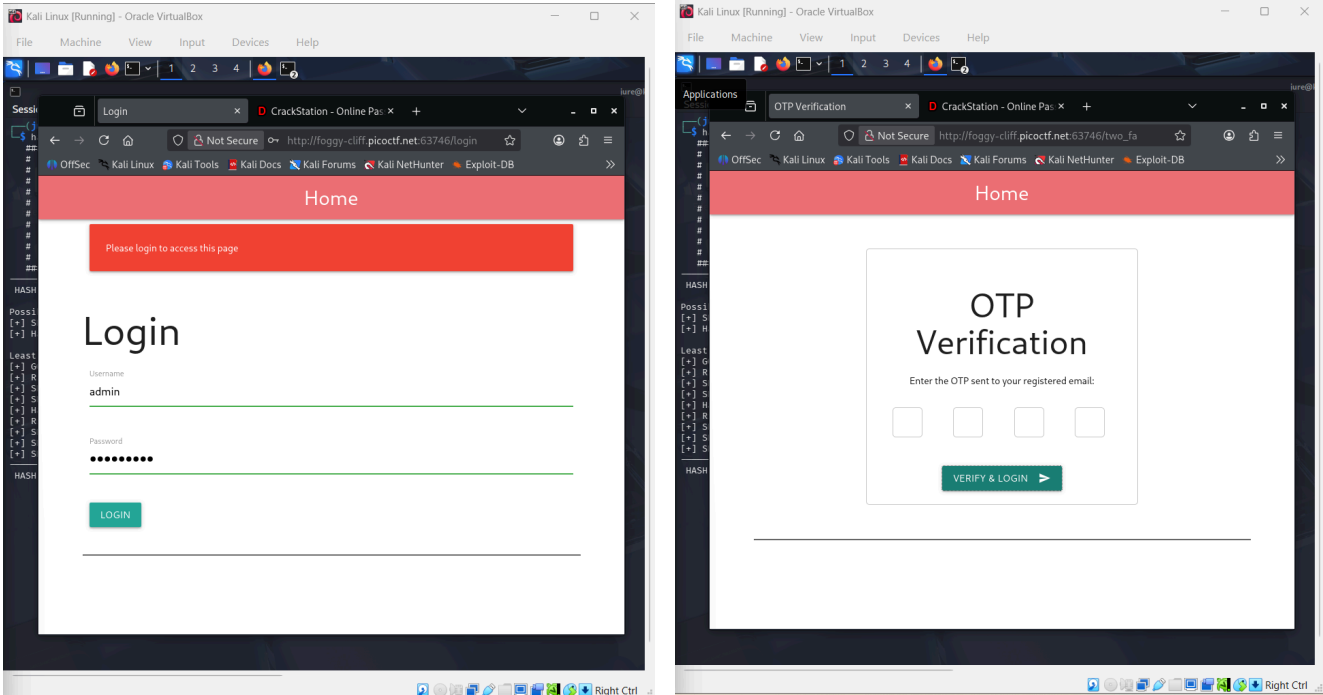
admin: c20fa16907343ee642d10f0b0db81bf629e6aaf6c906f26eabda079ca9e5ab67

```

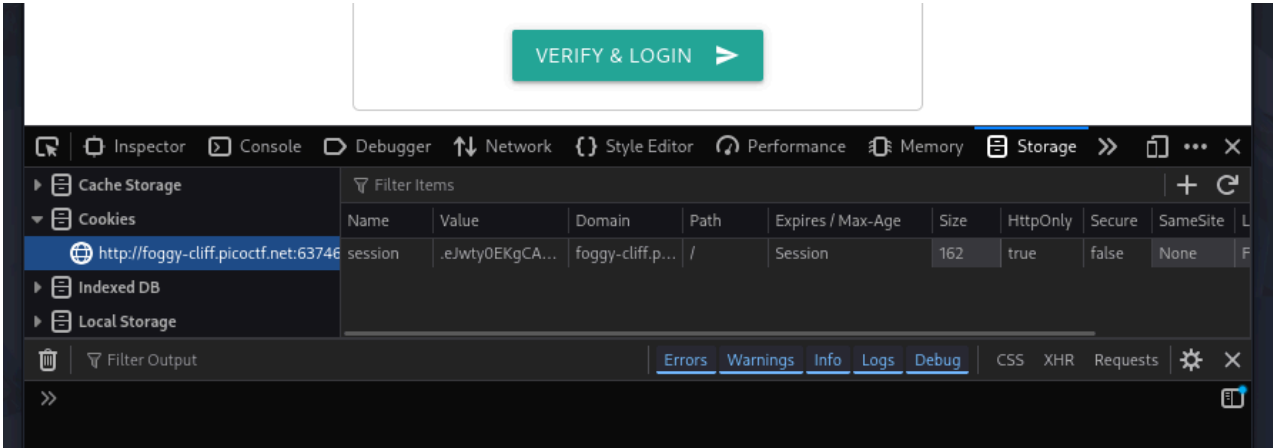
Next, the type of hash used for the password needed to be determined. A tool called **hash-identifier** in the kali linux cmd was used to analyze the hash format. The tool suggested that the hash was likely **SHA-256**. Instead of performing manual cracking with tools, an easier approach was used by submitting the hash to an online hash cracking service called **CrackStation**. Since the hint referenced the rockyou wordlist, it suggested that the password might be a commonly used one. After submitting the hash to CrackStation, the password was successfully recovered as **apple@123**.

The image shows a Kali Linux terminal window on the left and the CrackStation web interface on the right. The terminal displays the 'hash-identifier' tool output, identifying the input hash as SHA-256. The web interface shows the CrackStation homepage with a 'Free Password Hash Cracker' section. A sample hash is entered, and the tool returns a list of possible passwords, including 'apple@123'.

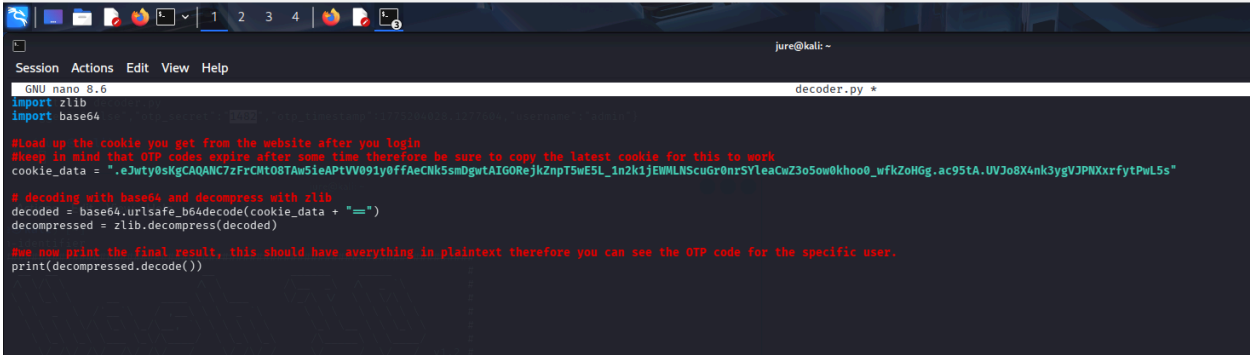
With the credentials obtained, the next step was to log in to the web application using the username `admin` and the recovered password `apple@123`. After logging in, the system redirected to a two-factor authentication (2FA) page that required an OTP. This meant that even though the password was correct, the login process still required a second verification step.

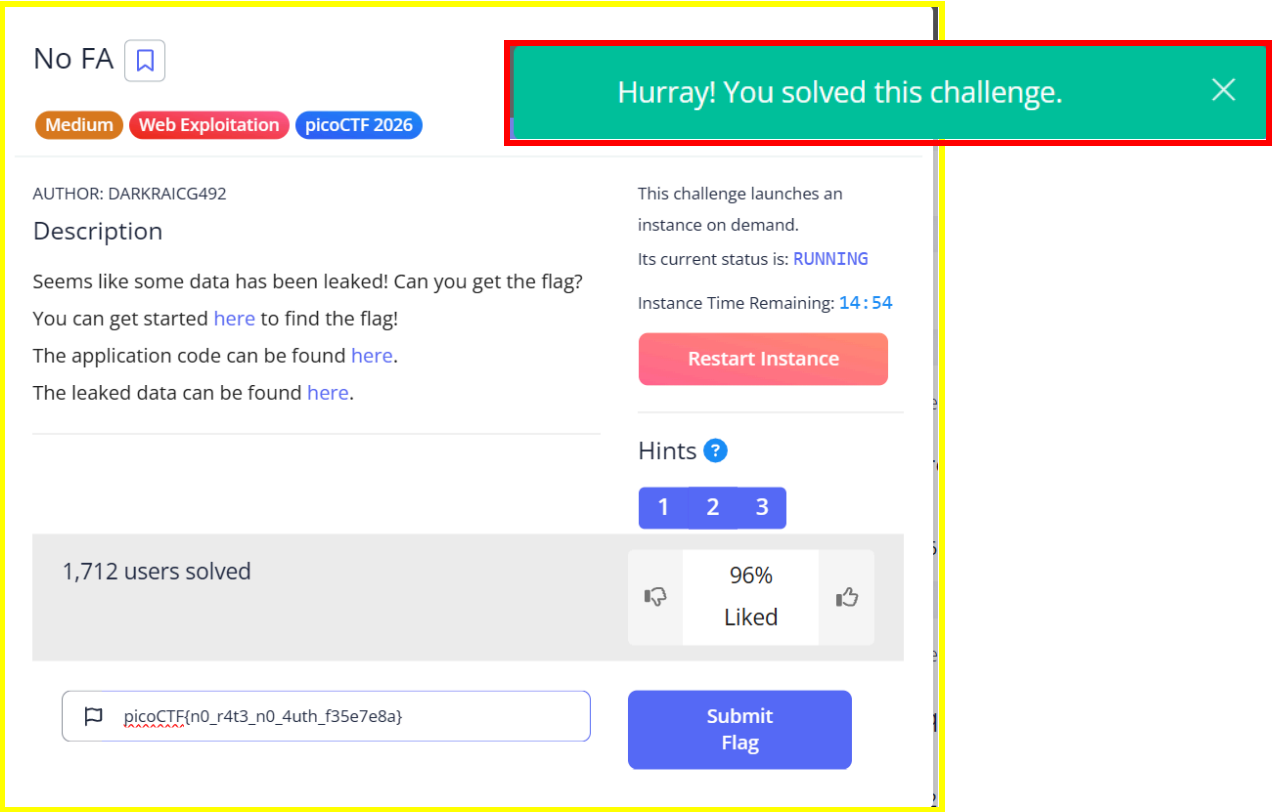
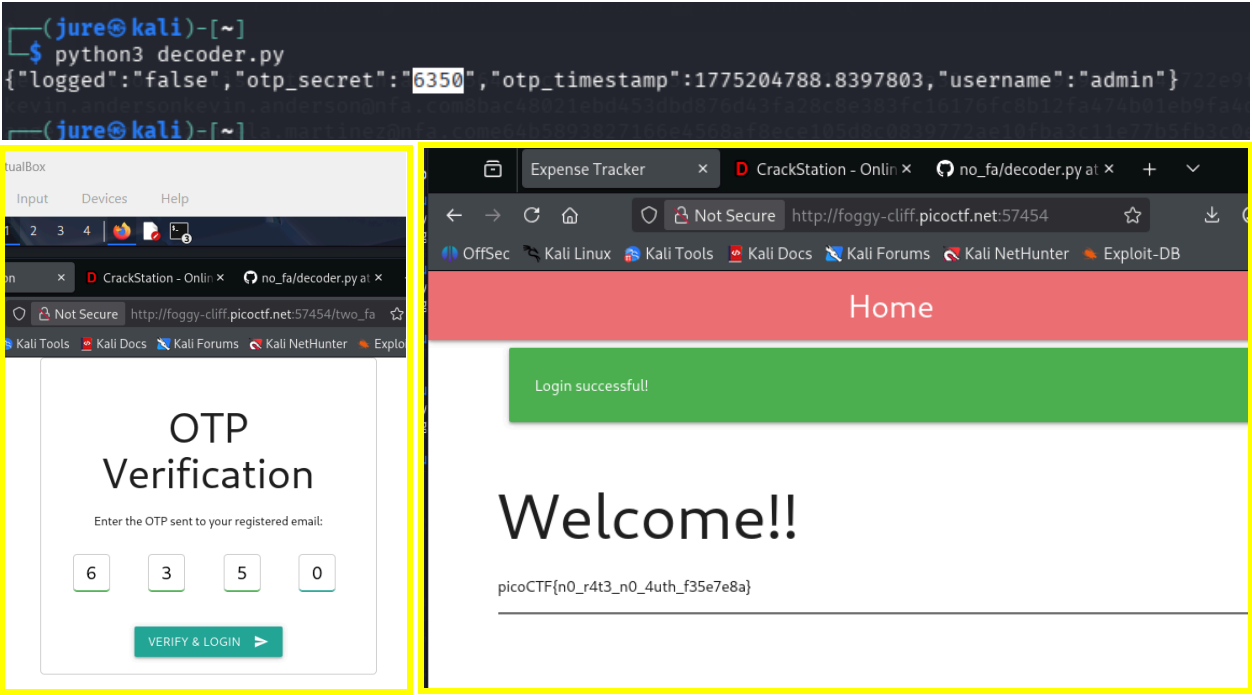


To bypass the 2FA mechanism, the browser's session cookies were inspected. A cookie stored by the server appeared to contain encoded data related to the session. Upon analysis, the cookie seemed to be Base64 encoded and also compressed, meaning it required both decoding and decompression to reveal its actual contents.



To handle the session cookie, I used a Python script named `decoder.py`, which I sourced from existing write-ups related to this particular challenge. The script performed two operations: it first decoded the Base64 data and then decompressed it using `zlib`. After running the script with the copied session cookie, the decoded output revealed internal session data, including the OTP secret value generated by the server. This OTP value represented the verification code required by the 2FA page. By quickly entering the extracted OTP into the verification field before it expired, the authentication process was completed successfully. Once the OTP was accepted, the application granted full access to the system and displayed the flag, completing the challenge.





Reflection:

This challenge showed me that two-factor authentication is only secure when implemented correctly. Even though the system used 2FA, storing sensitive data like the OTP secret inside a client-side cookie made it possible to decode and bypass the verification process. Overall, it helped me understand how weaknesses in password storage, hashing practices, and session management can compromise the entire authentication system.